MODULE 28 · DOMAIN 3 - DEPLOYMENT AND ORCHESTRATION

Orchestration and CI/CD

Pipelines, Step Functions, EventBridge, and what actually triggers a retrain.

MODULE 28 of 43 DOMAIN 3 - Deploy WEIGHT 24%

admin@sthithapragna.com

A manual pipeline works until the person who knows it is on holiday.

What an ML pipeline actually contains

01
Process
Prepare features. Modules 04 and 05.

›

02
Train
Or tune. Modules 14 and 15.

›

03
Evaluate
Score on held-out data. Module 17.

›

04
Gate
ConditionStep on the metric. Register only if it clears.

›

05
Register
Into the Model Registry, with metrics attached.

ON THE EXAM

Deployment is usually a separate pipeline. This one produces an approved model; another one ships it.

Pipeline step types

STEP	WHAT IT RUNS
ProcessingStep	A processing job. Preparation, evaluation, checks
TrainingStep	A training job
TuningStep	A hyperparameter tuning job. Module 15
TransformStep	A batch transform job
AutoMLStep	An Autopilot job. Module 13
ModelStep	Creates or registers a model
ConditionStep	Branches on a condition. The quality gate
LambdaStep	A Lambda function, for anything else
CallbackStep	Pauses for an external system to respond
FailStep	Fails the pipeline deliberately, with a message
QualityCheck and ClarifyCheck	Baselines for drift and bias. Modules 17 and 29
EMRStep	An EMR job, for large Spark work

Executions and monitoring

- Each run is an **execution**, with a status per step and the parameters it was given.
- Studio shows the **DAG visually**, with the failing step highlighted. The fastest diagnosis available.
- Each step links to the underlying job, so its CloudWatch logs are one click away.
- Execution status changes are **EventBridge events**, so alerting is wiring rather than code.
- A pipeline nobody watches fails silently. Alarm on failed executions from the first day.

> ON THE EXAM
>
> The visual DAG is genuinely useful: it shows which steps were cached, which ran, and where it stopped.

Selective execution

- Rerun **a subset of steps** from a previous execution, reusing its earlier outputs.
- So a failure at the evaluation step does not mean retraining from the beginning.
- Different from caching: caching skips unchanged steps, selective execution chooses which run.
- Useful for debugging a late step without paying for the early ones repeatedly.
- Combined with caching, most iteration costs only the part that changed.

> ON THE EXAM
>
> Caching is automatic and argument-based. Selective execution is deliberate and step-based. Both exist because rerunning everything is wasteful.

Two kinds of dependency

- **Data dependency:** reference another step's properties, and Pipelines wires the data through.
- Referencing a processing step's output as a training step's input **creates the ordering automatically**.
- **Custom dependency: add_depends_on** forces ordering with no data flowing between them.
- Use that when a step must finish first for a reason the data does not express.
- Together these define the DAG. You do not draw it; you declare the relationships.

> ON THE EXAM
>
> The DAG is inferred from what each step uses. That is why a pipeline diagram can surprise you when a reference is missing.

Property files, JsonGet, and the gate

- A **property file** registers an output file, such as module 17's evaluation.json.
- **JsonGet** reads a specific value out of it by path.
- A **ConditionStep** then compares that value to a threshold and branches.
- That is the whole mechanism behind gating registration on a quality metric.
- The path must match the report's structure exactly, or the condition cannot be evaluated.

> ON THE EXAM
>
> This is the concrete answer to module 17's 'gate registration on the metric'. Three pieces, and they are worth recognising by name.

Parameters

- A pipeline takes **parameters** supplied at execution time: data paths, instance types, thresholds.
- So one pipeline definition serves development, staging and production without being copied.
- The quality threshold as a parameter means raising the bar needs no code change.
- **Not all arguments can be parameterized**, and not all Python operations apply to parameters.
- That limitation is documented, and it catches people building elaborate dynamic pipelines.

ON THE EXAM

Parameters are what stop a pipeline being copy-pasted per environment, which is how three pipelines quietly diverge.

Testing a pipeline

- Run each step **standalone** first. A pipeline is a bad place to debug a processing script.
- Then run the pipeline on a **small sample**, with small instances, end to end.
- **Test the gate deliberately:** force a bad metric and confirm the ConditionStep blocks registration.
- A gate that has never been observed failing is a gate nobody has tested.
- Then run it on full data, once, before putting it on a trigger.

> ON THE EXAM
>
> The gate is the one thing that must be tested by making it fire. Everything else is tested by working.

Step caching

- **Caching configuration** lets a step reuse a previous successful run with the same arguments.
- So a pipeline rerun after changing only the training code **skips preprocessing entirely**.
- That turns iteration from tens of minutes into the part you actually changed.
- Cache hits depend on the arguments matching. A changed parameter invalidates the cache, correctly.
- Set an expiry, so a stale cached result cannot be reused indefinitely.

> ON THE EXAM
>
> Caching is the single biggest quality-of-life improvement in an ML pipeline, and it is off unless you configure it.

Retry policies, parallelism and failure

- **Retry policies** retry a step on specific errors, such as capacity or throttling.
- They do not fix a bug, from module 14. Retrying broken code three times costs three times as much.
- **Step parallelism** caps how many steps run at once, which protects shared quotas.
- **FailStep** ends the pipeline deliberately with a message, so a failure is explained rather than mysterious.
- **CallbackStep** pauses until an external system responds. That is how a human approval fits in.

> ON THE EXAM
>
> CallbackStep is the underrated one. It is how a pipeline waits for a person to approve something without polling.

SageMaker Pipelines or Step Functions?

PIPELINES, WHEN

The workflow is ML

- Training, tuning, evaluation, registration.
- Native lineage and experiment tracking.
- Steps are SageMaker AI jobs.
- Integrated with the Model Registry.

STEP FUNCTIONS, WHEN

The workflow is broader

- Glue, Lambda, ECS, SNS, and more.
- Complex branching, parallelism, error handling.
- ML is one part of a larger process.
- You already orchestrate this way.

THE TIEBREAKER

They are not exclusive. A common pattern is Step Functions orchestrating a broader process, invoking a SageMaker pipeline as one step.

What should actually trigger a retrain?

What can start a pipeline

TRIGGER	WHEN IT MAKES SENSE
Schedule	Data arrives predictably and the world changes slowly
New data arriving	EventBridge on S3. Retrain when there is something to learn from
Drift detected	Model Monitor raises an alarm. Module 29
Code or config change	CodePipeline on a commit. The CI/CD trigger
Manual	Still legitimate, especially early on. Just make it one click

ON THE EXAM

Drift-triggered retraining is the one worth aiming for: it retrains because something changed, not because it is Sunday.

Feature pipelines and training pipelines

- A **feature pipeline** keeps features current, writing to the Feature Store. Module 06.
- A **training pipeline** consumes those features and produces a model.
- They run on **different schedules**: features may update hourly, models monthly.
- Separating them means retraining reads current features without recomputing them.
- Combining them means features are only ever as fresh as the last retrain.

> ON THE EXAM
>
> Module 06's Feature Store is what makes this separation possible. Without it, feature freshness and model freshness share one schedule.

EventBridge as the connector

- **EventBridge** routes events from AWS services to targets, including a pipeline execution.
- It is what connects **something happened** to **something should run**, without polling.
- S3 object created, a Glue job finishing, a Model Monitor alarm, a schedule: all are events.
- Pipeline execution status is itself an event, so a completed pipeline can trigger the next thing.
- Modules 07, 08 and 29 all raise EventBridge. This is where those threads join.

> ON THE EXAM
>
> Almost every 'and then automatically' in this course is EventBridge underneath. It is the connective tissue of an ML platform.

Notifications on pipeline events

- Execution started, succeeded, failed, and step failed are all events you can act on.
- Route failures to a **channel a person reads**, not to an inbox nobody opens.
- A model registered in **PendingManualApproval** is worth notifying a reviewer about.
- Approval itself can be the manual gate, with the deploy pipeline waiting on it.
- Notify on failure and on approval needed. Notifying on every success trains people to ignore it.

> ON THE EXAM
>
> An automated pipeline removes the person who used to notice. Notifications are how you put the noticing back, deliberately.

MWAA and Airflow

- **Amazon Managed Workflows for Apache Airflow** runs Airflow as a managed service.
- The reason to use it: **the organisation already runs Airflow** and has the skills and DAGs.
- Airflow orchestrates data engineering broadly, so ML becomes one more DAG among many.
- It has no native Model Registry integration or ML lineage. You build that yourself.
- A reasonable pattern: Airflow triggers a SageMaker pipeline, and keeps the ML parts native.

> ON THE EXAM
>
> Existing investment is the legitimate reason to choose Airflow. Choosing it fresh for an ML workflow means rebuilding what Pipelines gives you.

CodePipeline, CodeBuild and the deployment pipeline

- **CodePipeline** orchestrates the software delivery: source, build, test, deploy.
- **CodeBuild** runs the build and test steps, including running your pipeline definition.
- There are usually **two pipelines**: one that **builds a model**, one that **deploys** it.
- The model-build pipeline ends at the registry. The deploy pipeline starts from an approved version.
- Model approval is the handover between them, and it can be manual or automated.

> ON THE EXAM
>
> Separating build from deploy is what lets a model be approved once and deployed to several environments.

SageMaker Projects

- Provisions a **complete MLOps setup** from a template: repositories, pipelines, and CI/CD.
- It gives a team the **two-pipeline structure** without designing it from scratch.
- Templates cover common patterns, including model build, model deploy, and third-party Git integration.
- Useful for standardising across teams, so every project has the same shape.
- The generated setup is a starting point. Most teams modify it substantially.

> ON THE EXAM
>
> The value is standardisation rather than the code: twenty teams with the same pipeline shape is easier to govern than twenty designs.

Multi-account deployment

- The common shape: **separate accounts** for development, staging and production.
- The model is built in one account and **deployed into others** from an approved registry entry.
- That gives a hard boundary: a development experiment cannot touch a production endpoint.
- Cross-account roles and a shared or replicated registry are what make it work.
- More setup, and it is what an auditor expects in a regulated organisation.

ON THE EXAM

Account boundaries are the strongest isolation AWS offers. A permissions mistake in development stops at the account edge.

Permissions for a pipeline

- A pipeline runs under an **execution role**, and each step's job may use its own role.
- That role is often over-permissioned, because it needs S3, ECR, training, and the registry.
- **Scope it to the specific buckets, images and resources** this pipeline uses, not to the account.
- Separate the **build** role from the **deploy** role. Building a model should not permit deploying one.
- Module 34 covers identity properly. This is where a pipeline's permissions bite.

ON THE EXAM

The pipeline role is a quiet risk: it can read training data, write models, and register them. Scope it deliberately.

Infrastructure as code

- **CloudFormation** or the **CDK** define endpoints, pipelines, roles and alarms as code.
- So an environment can be recreated, reviewed, and diffed, rather than clicked together.
- Module 24's guardrails and module 23's scaling policies belong here, not in a console.
- Console-created resources are the ones nobody can reproduce when they are needed elsewhere.
- The pipeline definition itself is code too, so it belongs in the same repository.

> ON THE EXAM
>
> If a safety control lives only in the console, it exists in one account and nobody reviewed it. Both are problems.

A pipeline is an artefact too

- The pipeline definition is **code in a repository**, reviewed like any other change.
- So a change to the quality threshold goes through review, rather than a console edit.
- Each execution records the definition it ran, so you can see what the pipeline looked like then.
- That matters for an audit: which pipeline, with which gates, produced this model?
- A console-edited pipeline has no diff and no reviewer. That is how a gate quietly gets lowered.

> ON THE EXAM
>
> The threshold in a ConditionStep is a governance control. Lowering it should look like a code change, because it is one.

Data versioning through a pipeline

- Record the **exact data version** the execution used, not just the bucket path.
- A path like **s3://bucket/data/** means something different every week. That is not a version.
- Use a dated prefix, a table snapshot, or a Feature Store point-in-time query. Module 06.
- Pass it as a **pipeline parameter**, so the execution record shows it.
- Without it, module 11's question cannot be answered: which data produced this model?

> ON THE EXAM
>
> An execution that read 'the latest data' recorded nothing useful. Versioning is what turns a rerun into a reproduction.

Automating the generative parts

- **Prompt versions** from module 12 are artefacts, so promoting one is a deployment step.
- **Knowledge base refresh cycles** should be scheduled, not remembered. Module 10's staleness risk.
- **Bedrock Evaluations** from module 18 can gate a prompt or model change, like a ConditionStep.
- An ingestion sync is an event, so EventBridge can run one when source documents change.
- Bedrock has no deployment guardrails, from module 25. The pipeline is where your safety comes from.

> ON THE EXAM
>
> The generative half of a platform needs the same automation as the classical half. It just has different artefacts.

Which orchestration

WHAT THE SCENARIO SAYS	THE ANSWER
Train, evaluate, register, with ML lineage	SageMaker Pipelines
Spans Glue, Lambda, ECS and ML	Step Functions
The team already runs Airflow	MWAA, possibly invoking a pipeline
Trigger on a commit	CodePipeline
Trigger when new data lands	EventBridge on S3
Trigger when the model degrades	Model Monitor alarm to EventBridge. Module 29
Standardise MLOps across many teams	SageMaker Projects
Recreate the whole environment	CloudFormation or CDK

ON THE EXAM

Two questions separate most of these: is the workflow ML-shaped, and what should make it run?

What to automate first

- **The gate.** A quality check that runs every time beats a fully automated pipeline without one.
- Then **evaluation**, so the gate has an honest metric to read.
- Then **training and preprocessing**, so the whole build is reproducible.
- Then **the trigger**, once you trust what happens when it fires.
- Automating deployment first, with no gate, is how a bad model reaches production unattended.

> ON THE EXAM
>
> Automate in the order that makes a mistake less likely, not in the order that removes the most manual work.

What automation costs, and saves

ITEM	EFFECT
Pipeline orchestration	Minimal. You pay for the jobs the steps run
Step caching	Real saving. Skipped steps cost nothing
Scheduled retraining	Full training cost, every schedule, whether needed or not
Triggered retraining	Training cost only when something actually changed
Step Functions	Per state transition. Cheap unless the workflow is enormous
MWAA	An always-on environment charge, unlike the others

ON THE EXAM

The orchestration is nearly free. The jobs it runs are not, which is why the trigger choice is a cost decision.

Quotas worth knowing about

QUOTA	NOTE
Pipelines per account	Capped, and raisable
Concurrent pipeline executions	Limits how many runs proceed at once
Steps per pipeline	A cap on pipeline size, which encourages splitting
Parallel steps	Your step parallelism setting, as well as quota
Underlying job quotas	Training and processing instance quotas still apply. Module 14

ON THE EXAM

The last row is the one that bites: a pipeline cannot exceed the training instance quota the account already had.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Every retrained model passes the gate	Gating on a training metric. Module 17
Pipeline reruns are slow and repetitive	Step caching not configured
ConditionStep cannot read the metric	Property file path does not match the report structure
Steps run in an unexpected order	Missing data dependency. Add an explicit add_depends_on
Pipeline retries a broken step repeatedly	Retry policy applied to a code fault, not a transient one
Model retrained weekly, never improves	Schedule trigger with no new data. Trigger on data or drift
Cannot reproduce last quarter's environment	Resources created in the console, not in code

ON THE EXAM

Row one is the most dangerous, because it looks like everything is working. An automated pipeline with a broken gate ships anything.

Automate the gates first. A pipeline with no quality gate just ships bad models faster.

> A team wants to automate a workflow that preprocesses data, trains an XGBoost model, evaluates it on a held-out set, and registers it only if AUC exceeds 0.85.
>
> They want the model's lineage and metrics recorded automatically, and the registry to record which dataset produced which model.

What should they build this with?

A SageMaker Pipelines, with a ConditionStep gating the ModelStep

B A Lambda function chain, each invoking the next on completion

C Step Functions, calling the SageMaker APIs directly

D MWAA, with an Airflow DAG calling boto3

◆ PAUSE AND DECIDE

What should they build this with?

A SageMaker Pipelines, with a ConditionStep gating the ModelStep

Every step is a SageMaker AI job, and lineage, metrics and registry integration come natively. The ConditionStep is the documented gate mechanism.

B A Lambda function chain, each invoking the next on completion

Rebuilds orchestration by hand, with no lineage, no retries, no caching, and a 15-minute limit on any step that does real work.

C Step Functions, calling the SageMaker APIs directly

Would work, and you lose native lineage and registry integration. Step Functions earns its place when the workflow extends beyond ML.

D MWAA, with an Airflow DAG calling boto3

Reasonable if the team already runs Airflow. Nothing in the scenario says they do, and it means building the ML integration yourself.

THE SIGNAL

The lineage and registry requirements are the deciding detail. They are native in Pipelines and self-built everywhere else.

> A retailer's nightly process extracts data with a Glue job, runs a Lambda to validate it, invokes an ECS task for a third-party enrichment API, then trains and deploys a model, then sends an SNS notification and updates a DynamoDB table.
>
> Several branches depend on outcomes, and failures must be handled differently per branch.

What should orchestrate this?

A SageMaker Pipelines, using LambdaStep for the non-ML parts

B EventBridge rules chained together, one per stage

C Step Functions, invoking a SageMaker pipeline for the ML portion

D A single large Lambda function coordinating everything

◆ PAUSE AND DECIDE

What should orchestrate this?

A SageMaker Pipelines, using LambdaStep for the non-ML parts

Forces Glue, ECS, SNS and DynamoDB through LambdaStep wrappers. Pipelines is built for ML steps, not as a general workflow engine.

B EventBridge rules chained together, one per stage

EventBridge routes events well and does not express branching, error handling or a visible overall state. There would be no single view of the workflow.

C Step Functions, invoking a SageMaker pipeline for the ML portion

Native integration with Glue, Lambda, ECS, SNS and DynamoDB, with per-branch error handling. The ML part stays in Pipelines and keeps its lineage.

D A single large Lambda function coordinating everything

One function owning a multi-service nightly workflow, inside a 15-minute limit, with hand-rolled error handling. It will not survive contact with production.

THE SIGNAL

The combination is the answer. Step Functions for breadth, a SageMaker pipeline for the ML, and each keeps what it is good at.

> A data scientist iterates on model code several times a day. Each pipeline run takes 38 minutes, of which 31 minutes is a preprocessing step whose inputs and parameters have not changed in three weeks.
>
> They want faster iteration without changing the pipeline's structure.

What should they enable?

A. Step parallelism, so preprocessing runs alongside training

B. Step caching, so unchanged steps reuse the previous successful run

C. A retry policy on the preprocessing step

D. A larger instance type for the preprocessing step

◆ PAUSE AND DECIDE

What should they enable?

A. Step parallelism, so preprocessing runs alongside training

Training depends on preprocessing's output, so they cannot run in parallel. Parallelism caps concurrency; it does not remove a dependency.

B. Step caching, so unchanged steps reuse the previous successful run

Inputs and parameters are unchanged, so the cache matches and the step is skipped entirely. Iteration drops to the seven minutes that actually changed.

C. A retry policy on the preprocessing step

Retries handle failures. The step is succeeding, just slowly and repeatedly.

D. A larger instance type for the preprocessing step

Makes a step faster that does not need to run at all. It costs more and still runs every time.

THE SIGNAL

Caching is off by default, which is why this situation is so common. Thirty-one minutes of identical work, every run, for three weeks.

> A team schedules their retraining pipeline to run every Sunday at 02:00. Six months in, they observe that most runs produce a model no better than the one in production, and occasionally slightly worse. Training cost is now a visible line item.
>
> They also had a period in March where the model degraded for eleven days before anyone noticed.

What should they change?

A. Run the schedule monthly instead of weekly, to cut cost

B. Keep the schedule and add a ConditionStep so worse models are not registered

C. Move to MWAA, which handles scheduling more flexibly

D. Trigger on new data and on drift detection, rather than on a schedule

◆ PAUSE AND DECIDE

What should they change?

A. Run the schedule monthly instead of weekly, to cut cost

Halves one problem and worsens the other. Eleven days of undetected degradation becomes potentially a month.

B. Keep the schedule and add a ConditionStep so worse models are not registered

A necessary gate they should have anyway, and it still pays for pointless training runs and still misses degradation between schedules.

C. Move to MWAA, which handles scheduling more flexibly

Changes the orchestrator while keeping the trigger that is causing both problems. A different scheduler is still a scheduler.

D. Trigger on new data and on drift detection, rather than on a schedule

The schedule fails in both directions: it retrains when nothing changed, and it does not retrain when something did. Event triggers fix both.

THE SIGNAL

Two symptoms, one cause. Wasted runs and eleven days of undetected degradation are both the schedule being unrelated to reality.

What to take from this module

- **Pipelines for ML workflows. Step Functions when it spans other services.** They combine.
- **Property file, JsonGet, ConditionStep** is the quality gate mechanism.
- **Step caching is off by default**, and it is the biggest iteration saving available.
- **A schedule fails in both directions.** Trigger on data arriving or on drift.
- **Two pipelines:** one builds a model to the registry, one deploys from an approved version.

ON THE EXAM

That closes Domain 3, at 24 percent of the exam. Module 29 opens Domain 4 with drift and monitoring.